

```

        int u = findset(ks[e[j]].u), v = findset(ks[e[j]].v);
        if (u != v) pa[u] = v;
    }
    int u = findset(ks[ES[i].newks].u), v = findset(ks[ES[i].newks].v);
    if (u != v) {        // 新的MST用 newks 替换 oldks
        ans++;
        pos[ES[i].newks]=oldpos, e[oldpos] = ES[i].newks, pos[ES[i].oldks] = 0;
    }
}
}
return ans;
}

int main() {
    for (int kase = 1; scanf("%d", &n) == 1; kase++) {
        for (int i = 0; i < n; i++) kp[i].read();
        make_segments(), make_events();
        int ans = solve();
        printf("Case %d: %d\n", kase, ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 11-14
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表7-3所示。

表7-3 生成树相关问题例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 7-19	POJ 3522	Slim Span	最小生成树	陈锋
例 7-20	UVa 11354	Bond	最小生成树；动态 LCA	陈锋
例 7-21	UVa 11865	Stream My Contest	最小有向生成树	刘汝佳
例 7-22	UVa 1279	Asteroid Rangers	动点的最小生成树	刘汝佳

7.4 二分图匹配

虽然很多问题都可以借助最大流或者最小费用流算法解决，但二分图匹配算法更简单，速度也更快。和前面的内容一样，算法思想本

身也是很有启发性的。

例7-23 【二分图最大匹配；最小点覆盖】我是SAM (SAM I AM, UVa 11419)

给出一个 $r \times c$ 大小的网格，网格上面放了 n 个 ($1 \leq r, c \leq 1000$, $1 \leq n \leq 10^6$) 目标。可以在网格外发射子弹，子弹会沿着垂直或者水平方向飞行，并且打掉飞行路径上的所有目标，如图7-5所示。你的任务是计算出最少需要多少子弹，各从哪些位置发射，才能把所有目标全部打掉。

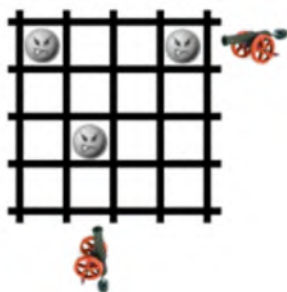


图7-5 射击

输出一行，其中第一个整数表示最少需要的子弹数目，接下来是这些子弹的发射位置。用 r_x 表示第 x 行， c_x 表示第 x 列。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;

const int maxn = 1000 + 5; // 单侧顶点的最大数目

// 二分图最大基数匹配
struct BPM {
    int n, m; // 左右顶点个数
    vector<int> G[maxn]; // 邻接表
    int left[maxn]; // left[i]为右边第 i 个点的匹配点编号, -1 表示不存在
    bool T[maxn]; // T[i]为右边第 i 个点是否已标记

    int right[maxn]; // 求最小覆盖数
    bool S[maxn]; // 求最小覆盖数

    void init(int n, int m) {
        this->n = n, this->m = m;
        for (int i = 0; i < n; i++) G[i].clear();
    }

    void AddEdge(int u, int v) { G[u].push_back(v); }

    bool match(int u) {
        S[u] = true;
        for (size_t i = 0; i < G[u].size(); i++) {
            int v = G[u][i];
            if (!T[v]) {
                T[v] = true;
                if (left[v] == -1 || match(left[v])) {
                    left[v] = u, right[u] = v;
                    return true;
                }
            }
        }
        return false;
    }
};

// 求最大匹配

```

```

int solve() {
    fill_n(left, m + 1, -1), fill_n(right, n + 1, -1);
    int ans = 0;
    for (int u = 0; u < n; u++) { // 从左边结点 u 开始增广
        fill_n(S, n + 1, false), fill_n(T, m + 1, false);
        if (match(u)) ans++;
    }
    return ans;
}

// 求最小覆盖, X 和 Y 为最小覆盖中的点集
int mincover(vector<int>& X, vector<int>& Y) {
    int ans = solve();
    fill_n(S, n + 1, false), fill_n(T, m + 1, false);
    for (int u = 0; u < n; u++) if (right[u] == -1) match(u); // 从所有 X 未盖点出发
增广
    for (int u = 0; u < n; u++) if (!S[u]) X.push_back(u); // X 中的未标记点
    for (int v = 0; v < m; v++) if (T[v]) Y.push_back(v); // Y 中的已标记点
    return ans;
}
};

BPM solver;
int main() {
    for (int R, C, N; scanf("%d%d%d", &R, &C, &N) == 3 && R && C && N;) {
        solver.init(R, C);
        for (int i = 0, r, c; i < N; i++)
            scanf("%d%d", &r, &c), solver.AddEdge(r - 1, c - 1);
        vector<int> X, Y;
        int ans = solver.mincover(X, Y);
        printf("%d", ans);
        for (size_t i = 0; i < X.size(); i++) printf(" r%d", X[i] + 1);
        for (size_t i = 0; i < Y.size(); i++) printf(" c%d", Y[i] + 1);
        printf("\n");
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 5.5.4 节例题 27
同类题目: Mission Improbable, Uva 1751
           Machine Schedule, HDU 1150
           Sentry Robots, ACM/ICPC SWERC 2012, Uva 12549
*/

```

例 7-24 【二分图最大独立集】保守的老师 (Guardian of Decency, NWERC 2005, POJ 2771)

Frank是一个思想有些保守的高中老师。有一次，他需要带 n ($n \leq 500$) 个学生出去旅行，但又怕其中一些学生在旅途中萌生爱意。为了降低这种事情发生的概率，他需要确保带出去的任意两个学生至少要满足下面4条中的一条。

- ☐ 身高至少相差40cm。
- ☐ 性别相同。
- ☐ 最喜欢的音乐属于不同类型。
- ☐ 最喜欢的体育比赛相同（他们很有可能是不同球队的球迷，这样他们就可能聊得不愉快）。

你的任务是帮Frank挑选尽量多的学生，使得任意两个学生至少满足上述条件中的一条。输出可以参加旅行的学生数目的最大值。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <vector>
#include <algorithm>
using namespace std;
const int maxn = 500 + 5; // 单侧顶点的最大数目
struct BPM { // 二分图最大基数匹配，邻接矩阵写法
    int n, m; // 左右顶点个数
    int G[maxn][maxn]; // 邻接表
    int left[maxn]; // left[i]为右边第 i 个点的匹配点编号，-1 表示不存在
    bool T[maxn]; // T[i]为右边第 i 个点是否已标记

    void init(int n, int m) {
        this->n = n, this->m = m;
        memset(G, 0, sizeof(G));
    }

    bool match(int u) {
        for (int v = 0; v < m; v++) if (G[u][v] && !T[v]) {
            T[v] = true;
            if (left[v] == -1 || match(left[v])) {
                left[v] = u;
                return true;
            }
        }
        return false;
    }

    // 求最大匹配
    int solve() {
        int ans = 0;
        fill_n(left, m + 1, -1);
        for (int u = 0; u < n; u++) { // 从左边结点 u 开始增广
            fill_n(T, m + 1, false);
            if (match(u)) ans++;
        }
        return ans;
    }
};

```

```

    }
};

BPM solver;

#include <iostream>
#include <string>
struct Student {
    int h;
    string music, sport;
    Student(int h = 0, const string& music = "", const string& sport = "")
        : h(h), music(music), sport(sport) {}
};

bool conflict(const Student& a, const Student& b) {
    return abs(a.h - b.h) <= 40 && a.music == b.music && a.sport != b.sport;
}

int main() {
    int T; cin >> T;
    for (int t = 0, n; cin >> n, t < T; t++) {
        vector<Student> male, female;
        Student s;
        for (int i = 0; i < n; i++) {
            string gender;
            cin >> s.h >> gender >> s.music >> s.sport;
            if (gender[0] == 'M') male.push_back(s);
            else female.push_back(s);
        }
        int x = male.size(), y = female.size();
        solver.init(x, y);
        for (int i = 0; i < x; i++)
            for (int j = 0; j < y; j++)
                if (conflict(male[i], female[j])) solver.G[i][j] = 1;
        printf("%d\n", x + y - solver.solve());
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.5.4 节例题 28
同类题目：Girls and Boys, HDU 1068
*/

```

例 7-25 【DAG 最小路径覆盖】出租车 (Taxi Cab Scheme, NWERC 2004, POJ 2060)

假设你在一座城市里负责一个大型活动的接待工作。明天将有 m ($0 \leq m < 500$) 位客人从城市的不同位置出发，到达他们各自的目的地。已知每人的出发时间、出发地点和目的地，你的任务是用尽量少的出租车接送他们，使得每次出租车接客人时，至少能提前一分钟到达客人所在的位置。注意，为了满足这一条件，要么这位客人是这辆出租车接送的第一个人，要么在接送完上一个客人后，有足够的时间从上一个目的地开到这位客人所在的位置。

为简单起见，假定城区是网格型的，地址用坐标 (x,y) 表示。出租车从 (x_1,y_1) 处到 (x_2,y_2) 处需要行驶 $|x_1-x_2|+|y_1-y_2|$ 分钟。输出最少需要的出租车数目。

【代码实现】


```

// 陈锋
#include <cstdio>
#include <cstring>
#include <vector>
#include <algorithm>
using namespace std;
const int maxn = 500 + 5; // 单侧顶点的最大数目
struct BPM { // 二分图最大基数匹配，邻接矩阵写法
    int n, m; // 左右顶点个数
    int G[maxn][maxn]; // 邻接表
    int left[maxn]; // left[i]为右边第 i 个点的匹配点编号，-1 表示不存在
    bool T[maxn]; // T[i]为右边第 i 个点是否已标记

    void init(int n, int m) {
        this->n = n, this->m = m;
        memset(G, 0, sizeof(G));
    }

    bool match(int u) {
        for (int v = 0; v < m; v++)
            if (G[u][v] && !T[v]) {
                T[v] = true;
                if (left[v] == -1 || match(left[v])) {
                    left[v] = u;
                    return true;
                }
            }
        return false;
    }

    int solve() { // 求最大匹配
        fill_n(left, m + 1, -1);
        int ans = 0;
        for (int u = 0; u < n; u++) { // 从左边结点 u 开始增广
            fill_n(T, m + 1, false);
            if (match(u)) ans++;
        }
        return ans;
    }
};

BPM solver;

int X1[maxn], Y1[maxn], X2[maxn], Y2[maxn], T1[maxn], T2[maxn];
inline int dist(int a, int b, int c, int d) { return abs(a - c) + abs(b - d); }

int main() {
    int T;
    scanf("%d", &T);
    for (int t = 0; n; t < T; t++) {
        scanf("%d", &n);
        for (int i = 0, h, m; i < n; i++) { // 注意输入时如何适应特定的格式
            scanf("%d:%d:%d:%d:%d", &h, &m, &X1[i], &Y1[i], &X2[i], &Y2[i]);
            T1[i] = h * 60 + m, T2[i] = T1[i] + dist(X1[i], Y1[i], X2[i], Y2[i]);
        }
        solver.init(n, n);
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                if (T2[i] + dist(X2[i], Y2[i], X1[j], Y1[j]) < T1[j])
                    solver.G[i][j] = 1; // 至少要提前 1 分钟到达
        printf("%d\n", n - solver.solve());
    }
    return 0;
}

/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.3.2 节例题 29
同类题目：The King's Problem, HDU 3861
*/

```

例7-26 【后继模型；二分图最小权匹配】巴士路线优化设计 (Optimal Bus Route Design, UVa 1349)

给出包含 n 个点 ($n \leq 100$) 的有向带权图，找到若干个有向圈，每个点恰好属于一个圈，要求权值和尽量小。注意，即使 (u,v) 和 (v,u) 都存在，它们的权值也不一定相同。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <cmath>
#include <algorithm>
using namespace std;

const int INF = 0x7f7f7f7f;
template<size_t SZ>
struct KM {
    int n;
    int W[SZ][SZ];           // 权值
    int Lx[SZ], Ly[SZ], slack[SZ]; // 顶标
    int left[SZ];            // left[i]为右边第 i 个点的匹配点编号，-1 表示不存在
    bool S[SZ], T[SZ];       // S[i]和 T[i]分别为左、右第 i 个点是否已标记

    void init(int n) {
```

```

    this->n = n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            W[i][j] = -INF;
}

void add_edge(int u, int v, int w) { W[u][v] = max(W[u][v], w); }

bool match(int i) {
    S[i] = true;
    for (int j = 0; j < n; j++) {
        if (T[j]) continue;
        int tmp = Lx[i] + Ly[j] - W[i][j];
        if (!tmp) {
            T[j] = true;
            if (left[j] == -1 || match(left[j])) {
                left[j] = i;
                return true;
            }
        } else
            slack[j] = min(slack[j], tmp);
    }
    return false;
}

void update() {
    int a = INF;
    for (int i = 0; i < n; i++)
        if (!T[i]) a = min(a, slack[i]);
    for (int i = 0; i < n; i++) {
        if (S[i]) Lx[i] -= a;
        if (T[i]) Ly[i] += a;
    }
}

void solve() {
    for (int i = 0; i < n; i++) {
        left[i] = -1;
        Lx[i] = -INF; Ly[i] = 0;
        for (int j = 0; j < n; j++)
            Lx[i] = max(Lx[i], W[i][j]);
    }
    for (int i = 0; i < n; i++) {
        fill_n(slack, n, INF);
        while (true) {
            fill_n(S, n, false), fill_n(T, n, false);
            if (match(i)) break; else update();
        }
    }
}

```

```

    }
};

const int NN = 104;
KM<NN> solver;
int main() {
    for (int n; scanf("%d", &n) == 1 && n;) {
        solver.init(n);
        for (int i = 0, j, c; i < n; i++) {
            while (scanf("%d", &j) && j)
                scanf("%d", &c), solver.add_edge(i, j - 1, -c);
        }
        int ans = 0;
        bool valid = true;
        solver.solve();
        for (int y = 0; y < n; y++) {
            int x = solver.left[y], w = solver.W[x][y];
            if (w == -INF) { valid = false; break; }
            ans += -w;
        }
        if (!valid) printf("N\n");
        else printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题11-10
同类题目: Game with Pearls, HDU 5090
*/

```

例7-27 【KM算法；可行顶标】少林决胜（Golden Tiger Claw, UVa 11383）

给定一个 $N \times N$ ($N \leq 500$) 矩阵，每个格子里都有一个正整数 $w(i,j)$ （均为不超过100的正整数）。你的任务是给每行确定一个整数 $row(i)$ ，每列也确定一个整数 $col(j)$ ，使得对于任意格子 (i,j) ， $w(i,j) \leq row(i) + col(j)$ 。所有 $row(i)$ 和 $col(j)$ 之和应尽量小。输出3行，第1行为从上到下各行的 $row(i)$ ；第2行为从左到右各列的 $col(j)$ ，第3行为所有 $row(i)$ 和 $col(j)$ 的总和的最小值。如果有多种方案，任意输出一种即可。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
template<size_t SZ>
struct KM {
    int n; // 左右顶点个数
    vector<int> G[SZ]; // 邻接表
    int W[SZ][SZ]; // 权值
};
```

```

int Lx[SZ], Ly[SZ];          // 顶标
int left[SZ];                // left[i]为右边第 i 个点的匹配点编号, -1 表示不存在
bool S[SZ], T[SZ];           // S[i]和T[i]分别为左、右第 i 个点是否已标记

void init(int n) {
    this->n = n;
    for (int i = 0; i < n; i++) G[i].clear();
    memset(W, 0, sizeof(W));
}

void AddEdge(int u, int v, int w) {
    G[u].push_back(v), W[u][v] = w;
}

bool match(int u) {
    S[u] = true;
    for (size_t i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        if (Lx[u] + Ly[v] == W[u][v] && !T[v]) {
            T[v] = true;
            if (left[v] == -1 || match(left[v])) {
                left[v] = u;
                return true;
            }
        }
    }
    return false;
}

void update() {
    int a = INF;
    for (int u = 0; u < n; u++) if (S[u])
        for (size_t i = 0; i < G[u].size(); i++) {
            int v = G[u][i];
            if (!T[v]) a = min(a, Lx[u] + Ly[v] - W[u][v]);
        }
    for (int i = 0; i < n; i++) {
        if (S[i]) Lx[i] -= a;
        if (T[i]) Ly[i] += a;
    }
}

void solve() {
    for (int i = 0; i < n; i++)
        Lx[i] = *max_element(W[i], W[i] + n), left[i] = -1, Ly[i] = 0;
    for (int u = 0; u < n; u++) {
        while (true) {
            for (int i = 0; i < n; i++) S[i] = T[i] = false;
            if (match(u)) break; else update();
        }
    }
}

```

```

    }
    }
}
};

const int maxn = 500 + 5; // 顶点的最大数目
KM<maxn> km;
int main() {
    for (int n, w; scanf("%d", &n) == 1; ) {
        km.init(n);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                scanf("%d", &w), km.AddEdge(i, j, w);
        km.solve();
        int sum = 0;
        for (int i = 0; i < n - 1; i++) printf("%d ", km.Lx[i]), sum += km.Lx[i];
        printf("%d\n", km.Lx[n - 1]);
        for (int i = 0; i < n - 1; i++) printf("%d ", km.Ly[i]), sum += km.Ly[i];
        printf("%d\n", km.Ly[n - 1]);
        printf("%d\n", sum + km.Lx[n - 1] + km.Ly[n - 1]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.5.2 节例题 24
*/

```

例7-28 【二分图最佳匹配】蚂蚁（Ants, NEERC 2008, POJ 3565)

给出 n ($1 \leq n \leq 100$) 个白点和 n 个黑点的坐标，且任意3点不共线。要求用 n 条不相交的线段把它们连接起来，其中每条线段恰好连接一个白点和一个黑点，每个点恰好连接到一条线段，如图7-6所示。输出 n 行，其中第 i 行为第 i 个白点所连接的黑点编号。

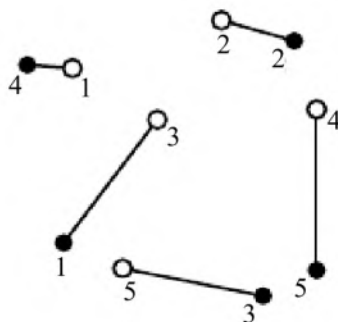


图7-6 线段相连

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <cmath>
#include <algorithm>
#include <cassert>
using namespace std;

const double INF = 1e30;
template<size_t SZ>
struct KM {
    double W[SZ][SZ];      // 权值
    double Lx[SZ], Ly[SZ]; // 顶标
```



```

int n, left[SZ];          // left[i]为右边第 i 个点的匹配点编号
bool S[SZ], T[SZ];       // S[i]和 T[i]分别为左、右第 i 个点是否已标记

bool eq(double a, double b) { return fabs(a - b) < 1e-9; }

void init(size_t _n) {
    assert(_n < SZ);
    this->n = _n;
}

bool match(int i) {
    S[i] = true;
    for (int j = 1; j <= n; j++) if (eq(Lx[i] + Ly[j], W[i][j]) && !T[j]) {
        T[j] = true;
        if (!left[j] || match(left[j])) {
            left[j] = i;
            return true;
        }
    }
    return false;
}

void update() {
    double a = INF;
    for (int i = 1; i <= n; i++) if (S[i])
        for (int j = 1; j <= n; j++) if (!T[j])
            a = min(a, Lx[i] + Ly[j] - W[i][j]);
    for (int i = 1; i <= n; i++) {
        if (S[i]) Lx[i] -= a;
        if (T[i]) Ly[i] += a;
    }
}

void solve() {
    for (int i = 1; i <= n; i++) {
        left[i] = Lx[i] = Ly[i] = 0;
        for (int j = 1; j <= n; j++)
            Lx[i] = max(Lx[i], W[i][j]);
    }
    for (int i = 1; i <= n; i++) {
        for (;;) {
            for (int j = 1; j <= n; j++) S[j] = T[j] = 0;
            if (match(i)) break; else update();
        }
    }
}

const int NN = 100 + 10;

```

```

int x1[NN], y1[NN], x2[NN], y2[NN];
KM<NN> solver;
int main() {
    for (int kase = 1, n; scanf("%d", &n) == 1; kase++) {
        if (kase > 1) printf("\n");
        solver.init(n);
        for (int i = 1; i <= n; i++) scanf("%d%d", &x1[i], &y1[i]);
        for (int i = 1; i <= n; i++) scanf("%d%d", &x2[i], &y2[i]);
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                solver.W[j][i] = -sqrt((double)(x1[i] - x2[j]) * (x1[i] - x2[j]) +
                    (double)(y1[i] - y2[j]) * (y1[i] - y2[j]));
        solver.solve(); // 最大权匹配
        for (int i = 1; i <= n; i++) printf("%d\n", solver.left[i]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.5.2 节例题 23
同类题目：Cyclic Tour, HDU 1853
*/

```

例7-29 【图论建模；二分图最佳匹配】固定分区内存管理 (Fixed Partition Memory Management, World Finals 2001, UVa 1006)

早期的多任务操作系统常把可用内存划分成一些大小固定的区域，不同的区域一般大小不同，而所有区域的大小之和即为可用内存的大小。给定一些程序，操作系统需要给每个程序分配一个区域，使得它们可以同时执行。可是每个程序的运行时间可能和它所占有的内存区域大小有关，因此调度并不容易。

编程计算最优的内存分配策略，即给定 m ($1 \leq m \leq 10$) 个区域的大小和 n ($1 \leq n \leq 50$) 个程序在 k ($k \leq 10$) 种内存大小范围下的运行时间，给出 k 对整数 $s_1, t_1, s_2, t_2, \dots, s_k, t_k$ ，满足 $s_i < s_{i+1}$ 。当内存不足 s_1 时，程序无法运行；当内存大小 s 满足 $s_i \leq s < s_{i+1}$ 时，运行时间为 t_i ；如果内存至少为 s_k ，运行时间为 t_k 。找出一种调度方案，使得平均回转时间（即平均结束时刻）尽量小。具体来说，你需要给每个

程序分配一个区域，使得任意两个程序都不会在同一时间运行在同一个内存区域中，而所有程序分配的区域大小都不小于该程序的最低内存需求。程序对内存的需求不会超过最大内存块的大小。

输出最小平均回转时间和调度方案。如果有多组解，任意输出一组即可。

【代码实现】

```
// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;
```

```

const int maxn = 500 + 5;      // 顶点的最大数目
const int INF = 1e9;

// 最大权匹配
struct KM {
    int n;                      // 左右顶点个数
    vector<int> G[maxn];        // 邻接表
    int W[maxn][maxn];          // 权值
    int Lx[maxn], Ly[maxn];     // 顶标
    int left[maxn];             // left[i]为右边第 i 个点的匹配点编号, -1 表示不存在
    bool S[maxn], T[maxn];      // S[i]和 T[i]分别为左、右第 i 个点是否已标记

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++) G[i].clear();
        memset(W, 0, sizeof(W));
    }

    void AddEdge(int u, int v, int w) { G[u].push_back(v), W[u][v] = w; }

    bool match(int u) {
        S[u] = true;
        for (int i = 0; i < G[u].size(); i++) {
            int v = G[u][i];
            if (Lx[u] + Ly[v] == W[u][v] && !T[v]) {
                T[v] = true;
                if (left[v] == -1 || match(left[v])) {
                    left[v] = u;
                    return true;
                }
            }
        }
        return false;
    }

    void update() {
        int a = INF;
        for (int u = 0; u < n; u++)
            if (S[u])
                for (int i = 0; i < G[u].size(); i++) {
                    int v = G[u][i];
                    if (!T[v]) a = min(a, Lx[u] + Ly[v] - W[u][v]);
                }
        for (int i = 0; i < n; i++) {
            if (S[i]) Lx[i] -= a;
            if (T[i]) Ly[i] += a;
        }
    }
}

```

```

void solve() {
    for (int i = 0; i < n; i++) {
        Lx[i] = *max_element(W[i], W[i] + n), left[i] = -1, Ly[i] = 0;
    }
    for (int u = 0; u < n; u++) {
        for (;;) {
            for (int i = 0; i < n; i++) S[i] = T[i] = false;
            if (match(u)) break;
            else update();
        }
    }
}

};

KM solver;

const int maxp = 50 + 5; // 程序 (program) 的最大数目
const int maxr = 10 + 5; // 区域 (region) 的最大数目
int n, m; // 程序数目和区域数目
int runtime[maxp][maxr]; // runtime[p][r] 为程序 p 在区域 r 中的运行时间

// 打印具体方案
void print_solution() {
    // 起始时刻、分配到的区域编号、总回转时间
    int start[maxp], region_number[maxp], total = 0;
    for (int r = 0; r < m; r++) {
        vector<int> ps; // 本 region 执行的所有程序, 逆序排列 ("倒数" 第 pos 个程序)
        for (int pos = 0; pos < n; pos++) {
            int right = r * n + pos, left = solver.left[right];
            if (left >= n) break; // 匹配到虚拟结点, 说明本 region 已经没有更多程序
            ps.push_back(left), region_number[left] = r;
            total -= solver.W[left][right]; // 权值取相反数
        }
        reverse(ps.begin(), ps.end());
        for (size_t i = 0, time = 0; i < ps.size(); i++)
            start[ps[i]] = time, time += runtime[ps[i]][r];
    }

    printf("Average turnaround time = %.2lf\n", (double)total / n);
    for (int p = 0; p < n; p++)
        printf("Program %d runs in region %d from %d to %d\n", p + 1,
            region_number[p] + 1, start[p],
            start[p] + runtime[p][region_number[p]]);
    printf("\n");
}

int main() {
    for (int kase = 1; scanf("%d%d", &m, &n) == 2 && m && n; kase++) {
        solver.init(m * n);
    }
}

```

```

int size[maxr];
for (int r = 0; r < m; r++) scanf("%d", &size[r]);
for (int p = 0; p < n; p++) {
    int s[10], t[10], k;
    scanf("%d", &k);
    for (int i = 0; i < k; i++) scanf("%d%d", &s[i], &t[i]);
    for (int r = 0; r < m; r++) {           // 计算程序 p 在内存区域 r 中的运行时间
        int& time = runtime[p][r];
        time = INF;
        if (size[r] < s[0]) continue;
        for (int i = 0; i < k; i++)
            if (i == k - 1 || size[r] < s[i + 1]) {
                time = t[i];
                break;
            }
        // 连边 X(p) -> Y(r,pos)
        for (int pos = 0; pos < n; pos++)    // 要求最小值，权值要取相反数
            solver.AddEdge(p, r * n + pos, -(pos + 1) * time);
    }
}
for (int i = n; i < n * m; i++)           // 补完其他边
    for (int j = 0; j < n * m; j++) solver.AddEdge(i, j, 1);
solver.solve();
printf("Case %d\n", kase);
print_solution();
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.5.2 节例题 25
同类题目：Going Home, POJ 2195
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表7-4所示。

表7-4 二分图匹配例题归纳